

一、概述

模块支持 Modbus-RTU 通讯协议。本手册详细介绍了 Modbus-RTU 协议及其寄存器映射和高级命令。

模块将全部的功能都映射三种类型的 Modbus 对象（寄存器）上，如果使用主机支持 Modbus-RTU 协议，就可以直接操作这些对象，不用关注协议实现的细节（第二节所述内容）。

对象类型	对象类型	访问类型	作用
离散量	单个比特	只读	读取模块状态标志
线圈	单个比特	读写	控制模块执行功能
保持寄存器	16-比特字	读写	读取重量等信息

二、Modbu-RTU 协议简介

1、帧格式

在 RTU 模式，报文帧由时长至少为 3.5 个字符时间的空闲间隔区分。在后续的部分，这个时间区间被称作 t3.5。

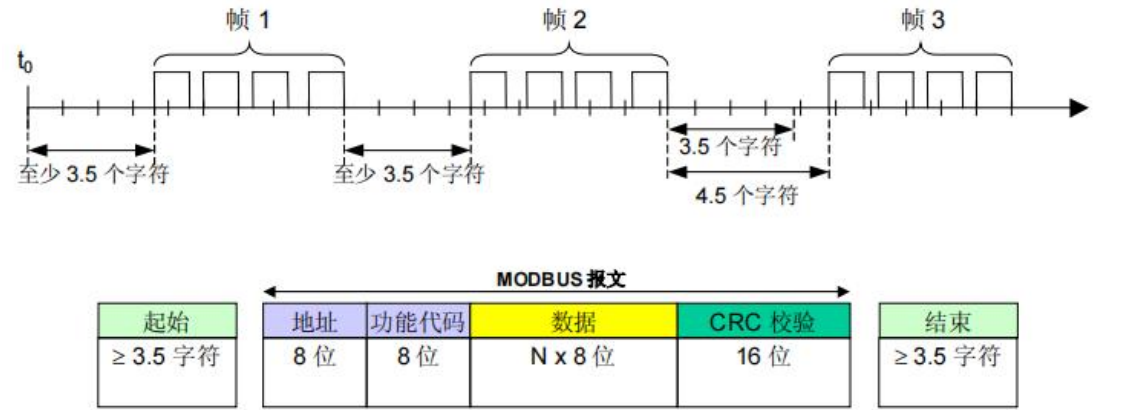
例如：

$3.5t(9600) = 1000 * 11 * 3.5 / 9600 \approx 4ms$

$3.5t(19200) = 1000 * 11 * 3.5 / 38400 \approx 2ms$

$3.5t(>19200)$ Modbus 协议规定统一为 1.75ms

对于某些主机（如 PC），如果实现不了精确定时，只需将帧间隔判断时间设置为大于 t3.5 即可。



2、CRC 校验

在 RTU 模式中，CRC 检验整个报文的内容（包括“地址”、“功能代码”和“数据”）。附加在报文之后时，首先附加低字节，然后是高字节。CRC 高字节为报文发送的最后一个字节。

CRC 生成 C 语言函数如下：

```
unsigned short CRC16 ( puchMsg, usDataLen ) /* 函数以 unsigned short 类型返回 CRC */
unsigned char *puchMsg ; /* 用于计算 CRC 的报文 */
```

```

unsigned short usDataLen ;                /* 报文中的字节数 */
{
unsigned char uchCRCHi = 0xFF ;           /* CRC 的高字节初始化 */
unsigned char uchCRCLo = 0xFF ;          /* CRC 的低字节初始化 */
unsigned uIndex ;                         /* CRC 查询表索引 */
while (usDataLen-- )                     /* 完成整个报文缓冲区 */
{
uIndex = uchCRCLo ^ *puchMsgg++ ;        /* 计算 CRC */
uchCRCLo = uchCRCHi ^ auchCRCHi[uIndex] ;
uchCRCHi = auchCRCLo[uIndex] ;
}
return (uchCRCHi << 8 | uchCRCLo) ;
}

```

高字节表

/* 高位字节的 CRC 值 */

```

static unsigned char auchCRCHi[] = {
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81,
0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81,
0x40, 0x01, 0xC0,
0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1,
0x81, 0x40, 0x01,
0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01,
0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
0x00, 0xC1, 0x81,
0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80,
0x41, 0x01, 0xC0,
0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
0x80, 0x41, 0x01,
0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81,
0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,
0x40, 0x01, 0xC0,

```

```

0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
0x80, 0x41, 0x01,
0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81,
0x40
};

```

低字节表

/* 低位字节的 CRC 值 */

```

static char auchCRCLo[] = {
0x00, 0xC0, 0xC1, 0x01, 0xC3, 0x03, 0x02, 0xC2, 0xC6, 0x06, 0x07, 0xC7,
0x05, 0xC5, 0xC4,
0x04, 0xCC, 0x0C, 0x0D, 0xCD, 0x0F, 0xCF, 0xCE, 0x0E, 0x0A, 0xCA, 0xCB,
0x0B, 0xC9, 0x09,
0x08, 0xC8, 0xD8, 0x18, 0x19, 0xD9, 0x1B, 0xDB, 0xDA, 0x1A, 0x1E, 0xDE,
0xDF, 0x1F, 0xDD,
0x1D, 0x1C, 0xDC, 0x14, 0xD4, 0xD5, 0x15, 0xD7, 0x17, 0x16, 0xD6, 0xD2,
0x12, 0x13, 0xD3,
0x11, 0xD1, 0xD0, 0x10, 0xF0, 0x30, 0x31, 0xF1, 0x33, 0xF3, 0xF2, 0x32,
0x36, 0xF6, 0xF7,
0x37, 0xF5, 0x35, 0x34, 0xF4, 0x3C, 0xFC, 0xFD, 0x3D, 0xFF, 0x3F, 0x3E,
0xFE, 0xFA, 0x3A,
0x3B, 0xFB, 0x39, 0xF9, 0xF8, 0x38, 0x28, 0xE8, 0xE9, 0x29, 0xEB, 0x2B,
0x2A, 0xEA, 0xEE,
0x2E, 0x2F, 0xEF, 0x2D, 0xED, 0xEC, 0x2C, 0xE4, 0x24, 0x25, 0xE5, 0x27,
0xE7, 0xE6, 0x26,
0x22, 0xE2, 0xE3, 0x23, 0xE1, 0x21, 0x20, 0xE0, 0xA0, 0x60, 0x61, 0xA1,
0x63, 0xA3, 0xA2,
0x62, 0x66, 0xA6, 0xA7, 0x67, 0xA5, 0x65, 0x64, 0xA4, 0x6C, 0xAC, 0xAD,
0x6D, 0xAF, 0x6F,
0x6E, 0xAE, 0xAA, 0x6A, 0x6B, 0xAB, 0x69, 0xA9, 0xA8, 0x68, 0x78, 0xB8,
0xB9, 0x79, 0xBB,
0x7B, 0x7A, 0xBA, 0xBE, 0x7E, 0x7F, 0xBF, 0x7D, 0xBD, 0xBC, 0x7C, 0xB4,
0x74, 0x75, 0xB5,
0x77, 0xB7, 0xB6, 0x76, 0x72, 0xB2, 0xB3, 0x73, 0xB1, 0x71, 0x70, 0xB0,
0x50, 0x90, 0x91,
0x51, 0x93, 0x53, 0x52, 0x92, 0x96, 0x56, 0x57, 0x97, 0x55, 0x95, 0x94,
0x54, 0x9C, 0x5C,
0x5D, 0x9D, 0x5F, 0x9F, 0x9E, 0x5E, 0x5A, 0x9A, 0x9B, 0x5B, 0x99, 0x59,
0x58, 0x98, 0x88,
0x48, 0x49, 0x89, 0x4B, 0x8B, 0x8A, 0x4A, 0x4E, 0x8E, 0x8F, 0x4F, 0x8D,
0x4D, 0x4C, 0x8C,
0x44, 0x84, 0x85, 0x45, 0x87, 0x47, 0x46, 0x86, 0x82, 0x42, 0x43, 0x83,

```

```
0x41, 0x81, 0x80,  
0x40  
};
```

3、通道地址

通道地址范围（即通讯地址）1~247，地址 0x01 为默认通道地址，用于模块的寻址，模块只对地址相符的命令执行动作和应答。在一个总线上，每个模块都有唯一的地址，主机通过不同的地址来区别地访问各个模块。

4、功能代码

功能代码（简称功能码）决定了请求的类型，也决定了数据字段的含义。不同功能码的请求帧和响应帧中数据字段的含义不尽相同，见下述各个功能码的说明。

访问模式	功能	功能码(十进制)
比特访问	读线圈	01
	写单个线圈	05
16 比特访问	读多个寄存器	03
	写单个寄存器	06
	写多个寄存器	16

5、01(0x01)读线圈

请求 PDU

功能码	1 个字节	0x01
起始地址	2 个字节	0x0000 至 0xFFFF
线圈数量	2 个字节	1 至 2000 (0x7D0)

响应 PDU

功能码	1 个字节	0x01
起始地址	2 个字节	N*
线圈状态	N 个字节	n=N 或 N+1

*N=输出数量/8，如果余数不等于 0，那么 N = N+1
如果返回的输出数量不是八的倍数，将用零填充最后数据字节中的剩余比特（一直到字节的高位端）。字节数量域说明了数据的完整字节数。

这是一个请求读线圈输出 20-38 的实例：

请求		响应	
域名	(十六进制)	域名	(十六进制)
功能	01	功能	01
起始地址 Hi	00	字节数	03
起始地址 Lo	13	输出状态 27-20	CD
输出数量 Hi	00	输出状态 35-28	6B
输出数量 Lo	13	输出状态 38-36	05

6、03(0x03)读保持寄存器

请求

功能码	1 个字节	0x03
起始地址	2 个字节	0x0000 至 0xFFFF
寄存器数量	2 个字节	1 至 125 (0x7D)

响应

功能码	1 个字节	0x03
字节数	1 个字节	2*N*
寄存器值	N*2 个字节	

*N＝寄存器的数量

这是一个请求读寄存器 108-110 的实例：

请求		响应	
域名	(十六进制)	域名	(十六进制)
功能	03	功能	03
起始地址 Hi	00	字节数	06
起始地址 Lo	6B	寄存器值 Hi (108)	02
高寄存器编号	00	寄存器值 Lo (108)	2B
低寄存器编号	03	寄存器值 Hi (109)	00
		寄存器值 Lo (109)	00
		寄存器值 Hi (110)	00
		寄存器值 Lo (110)	64

7、05(0x05)写单个线圈

请求

功能码	1 个字节	0x05
输出地址	2 个字节	0x0000 至 0xFFFF
输出值	2 个字节	0x00000 至 0xFF00

响应

功能码	1 个字节	0x05
输出地址	2 个字节	0x0000 至 0xFFFF
输出值	2 个字节	0x00000 至 0xFF00

这是一个请求写线圈 173 为 ON 的实例：

请求		响应	
域名	(十六进制)	域名	(十六进制)
功能	05	功能	05
输出地址 Hi	00	输出地址 Hi	00
输出地址 Lo	AC	输出地址 Lo	AC
输出值 Hi	FF	输出值 Hi	FF

输出值 Lo	00	输出值 Lo	00
--------	----	--------	----

8、06(0x06)写单个寄存器

请求

功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0000 至 0xFFFF
寄存器值	2 个字节	0x00000 至 0xFFFF

响应

功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0000 至 0xFFFF
寄存器值	2 个字节	0x00000 至 0xFFFF

这是一个请求将十六进制 00 03 写入寄存器 2 的实例：

请求		响应	
域名	(十六进制)	域名	(十六进制)
功能	06	功能	06
寄存器地址 Hi	00	寄存器地址 Hi	00
寄存器地址 Lo	01	寄存器地址 Lo	01
输出值 Hi	00	输出值 Hi	00
输出值 Lo	03	输出值 Lo	03

9、16(0x10)写多个寄存器

请求 PDU

功能码	1 个字节	0x10
起始地址	2 个字节	0x0000 至 0xFFFF
寄存器数量	2 个字节	0x0001 至 0x0078
字节数	1 个字节	2*N*
寄存器值	N*2 个字节*	值

*N=寄存器的数量

响应 PDU

功能码	1 个字节	0x10
起始地址	2 个字节	0x0000 至 0xFFFF
寄存器数量	2 个字节	1 至 123 (0x7B)

这是一个请求将十六进制 00 0A 和 01 02 写入以 2 开始的两个寄存器的实例：

请求		响应	
域名	(十六进制)	域名	(十六进制)
功能	10	功能	10
起始地址 Hi	00	起始地址 Hi	00
起始地址 Lo	01	起始地址 Lo	01
寄存器数量 Hi	00	寄存器数量 Hi	00

寄存器数量 Lo	02	寄存器数量 Lo	02
字节数	04		
寄存器值 Hi	00		
寄存器值 Lo	0A		
寄存器值 Hi	01		
寄存器值 Lo	02		

10、MODBUS 异常响应

当客户机设备向服务器设备发送请求时，客户机希望一个正常响应。从主站询问中出现下列四种可能事件之一：

- 如果服务器设备接收到无通信错误的请求，并且可以正常地处理询问，那么服务器设备将返回一个正常响应。
- 如果由于通信错误，服务器没有接收到请求，那么不能返回响应。客户机程序将最终处理请求的超时状态。
- 如果服务器接收到请求，但是检测到一个通信错误（奇偶校验、LRC、CRC、...），那么不能返回响应。客户机程序将最终处理请求的超时状态。
- 如果服务器接收到无通信错误的请求，但不能处理这个请求（例如，如果请求读一个不存在的输出或寄存器），服务器将返回一个异常响应，通知用户错误的本质特性。

异常响应报文有两个与正常响应不同的域：

功能码域：在正常响应中，服务器利用响应功能码域来应答最初请求的功能码。所有功能码的最高有效位（MSB）都为 0（它们的值都低于十六进制 80）。在异常响应中，服务器设置功能码的 MSB 为 1。这使得异常响应中的功能码值比正常响应中的功能码值高十六进制 80。

通过设置功能码的 MSB，客户机的应用程序能够识别异常响应，并且能够检测异常码的数据域。

数据域：在正常响应中，服务器可以返回数据域中数据或统计表（请求中要求的任何报文）。在异常响应中，服务器返回数据域中的异常码。这就定义了产生异常的服务器状态。

异常码列表：

MODBUS 异常码		
代码	名称	含义
02	非法数据地址	对于本仪表来说，该错误码代表所接收到的数据地址是不允许的地址。
03	非法数据值	写入的数据不符合允许的范围。
04	从站设备故障	当仪表正在试图执行请求的操作时，产生不可恢复的差错。
07	不成功的编程请求	对仪表来说，所接收到的命令在当前条件下无法执行。

11、参考资料

详细的协议请参考以下资料：

《Modbus 应用协议》（国标 GB/T19582.1-2008）

《Modbus 协议在串行链路上的实现指南》（国标 GB/T19582.2-2008）

三、读保持寄存器

1、概述

保持寄存器以字（16bits）为单位，可读取（功能码 0x03）。通过读取保持寄存器可以获得模块的重据和状态。

寄存器地址 (十进制)	字数 (16bits)	功能说明	备注
0	2	实时重量	补码表示，高位在前
2	1	状态标志	
22	2	微伏值	补码表示，高位在前
9000	2	版本序号	
9002	2	开发时间	

2、实时重量(0)

实时重量为 32bit 补码占两个字。

两个字补码的表示举例：

数值	高字	低字
123456	0x0001	0xE240
32768	0x0000	0x8000
32767	0x0000	0x7fff
12345	0x0000	0x3049
1	0x0000	0x0001
0	0x0000	0x0000
-1	0xffff	0xffff
-123456	0xfffe	0x1dc0

请求

地址通道地址	1 个字节	0x01-0xF8
功能码	1 个字节	0x03
起始地址	2 个字节	0x0000
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x03
字节数	1 个字节	0x04
寄存器值	N*2 个字节	高位在前，低位在后
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个读取通道地址 (0x01) 重量值的实例：

请求：01 03 00 00 00 02 C4 0B

响应：01 03 04 00 00 00 01 3B F3(重量值： 1)

3、状态标志(2)

bit15~4	bit3	bit2	bit1	bit0
0	负数	零位	超载	稳定

bit0: 稳定

重量变化小于判稳范围后，稳定标志位为 1；

bit1:超载

当重量值大于满量程加 9 个分度值时，超载标志位为 1。当超载标志为 1 时，主机必须停止显示重量，并做出相应的报警提醒用户，避免压坏传感器；

bit2:零位

当重量值等于 0 时，零位标志位为 1；

bit3:负数

当重量值为负数时，负数标志位为 1；

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x03
起始地址	2 个字节	0x0002
寄存器数量	2 个字节	0x0001
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x03
字节数	1 个字节	0x02
寄存器值	N*2 个字节	--
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个读取通道地址 (0x01) 状态的实例：

请求：01 03 00 00 00 03 05 CB

响应：01 03 06 00 00 00 01 00 01 B1 75(重量值:1,状态:稳定)

4、微伏值(22)

毫伏值为 32bit 补码占两个字。

请求

地址通道地址	1 个字节	0x01-0xF8
功能码	1 个字节	0x03
起始地址	2 个字节	0x0016
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x03
字节数	1 个字节	0x04

寄存器值	N*2 个字节	高位在前，低位在后
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个读取通道地址 (0x01) 微伏值的实例：

请求：01 03 00 16 00 02 25 CF

响应：01 03 04 00 00 00 32 7B E6(毫伏值：50uV)

5、版本序号(9000)

记录产品版本序号。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x03
起始地址	2 个字节	0x2328
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x03
字节数	1 个字节	0x04
寄存器值	N*2 个字节	--
CRC16 校验码	2 个字节	低位在前，高位在后

如显示 0x00190107，格式 XX XXXX，版本号为 25. 01. 07。

请求：01 03 23 28 00 02 4F 87

响应：01 03 04 00 19 01 07 6B A6

6、开发时间(9002)

记录产品开发时间。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x03
起始地址	2 个字节	0x232A
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x03
字节数	1 个字节	0x04
寄存器值	N*2 个字节	--
CRC16 校验码	2 个字节	低位在前，高位在后

如显示 0x00190107，则开发时间是 2025 年 1 月 7 日。

请求：01 03 23 2A 00 02 EE 47

响应：01 03 04 00 19 01 07 6B A6F

四、写保持寄存器

1、概述

保持寄存器以字（16bit）为单位，可写（功能码 0x06、0x10）。主要用于配置参数，和执行某些复杂的指令。

注：寄存器大小占两个字的保持寄存器不能通过 0x06 命令配置。

寄存器地址 (十进制)	字数 (16bits)	功能说明	备注
6	1	置零	写入非零值则清零
7	1	上电自动清零开关	
8	1	零点跟踪范围	
9	1	判稳范围	
10	1	开机置零范围	
12	1	稳态滤波级数	
13	1	AD 采样速度	
14	1	通道号	
15	1	波特率	
16	1	通信格式	
17	1	通信模式	
18	1	小数点位置	
19	1	最小分度值	
20	2	最大量程	最大量程 $\leq$ 最小分度 *100000
22	2	有砝码零点标定	写入非零值
30	2	有砝码增益标定第一点	
32	2	有砝码增益标定第二点	第一点 $\leq$ 输入砝码重量 $\leq$ 最大量程
34	2	有砝码增益标定第三点	第二点 $\leq$ 输入砝码重量 $\leq$ 最大量程
36	2	有砝码增益标定第四点	第三点 $\leq$ 输入砝码重量 $\leq$ 最大量程

2、置零(6)

写入非零值，模块执行置零功能。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0006
寄存器值	2 个字节	0x00001 至 0xFFFF
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06/0x10
寄存器地址	2 个字节	0x0006
寄存器值	2 个字节	0x00001 至 0xFFFF
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 置零的实例：

请求：01 06 00 06 00 01 A8 0B

响应：01 06 00 06 00 01 A8 0B

3、上电自动清零开关(7)

模块重新上电后，是否以当前重量作为零点，1：开启上电自动清零；0：关闭上电自动清零。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0007
寄存器值	2 个字节	0x00000 至 0xFFFF
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0007
寄存器值	2 个字节	0x00001 至 0xFFFF
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 开启上电自动清零的实例：

请求：01 06 00 07 00 01 F9 CB

响应：01 06 00 07 00 01 F9 CB

5、零点跟踪范围(8)

“d” 为分度值, 当重量处于零点跟踪范围，模块逐步修正零点，使输出趋向于零。

数值	0	1	2	3	4	5	6	7	8	9
范围	不跟踪	1d	2d	3d	4d	5d	6d	7d	8d	9d

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0008
寄存器值	2 个字节	0x0000 至 0x0009
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0008
寄存器值	2 个字节	0x0000 至 0x0009
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 零点跟踪设置为 2d 的实例：

请求：01 06 00 08 00 02 89 C9

响应：01 06 00 08 00 02 89 C9

6、判稳范围(9)

“d” 为分度值。当重量的变化小于设定的判稳范围 1.6 秒后，状态字中稳定位置的置位。

数值	1	2	3	4	5	6	7	8	9
范围	1d	2d	3d	4d	5d	6d	7d	8d	9d

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0009
寄存器值	2 个字节	0x0001 至 0x0009
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0009
寄存器值	2 个字节	0x0001 至 0x0009
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 判稳范围设置为 9d 的实例：

请求：01 06 00 09 00 09 99 CE

响应：01 06 00 09 00 09 99 CE

7、开机置零范围(10)

开机置零范围（0~99），单位为 FULL%，例如满量程为 500kg，开机置零范围为 20，那么开机置零的范围值就是±100kg，模块重新上电后，如果毛重小于±100kg，模块就把该重量作为零点，输出毛重为 0。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x000A
寄存器值	2 个字节	0x0000 至 0x0063

CRC16 校验码	2 个字节	低位在前，高位在后
-----------	-------	-----------

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x000A
寄存器值	2 个字节	0x0000 至 0x0063
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 开机置零范围设置为 50 的实例：

请求：01 06 00 0A 00 32 28 1D

响应：01 06 00 0A 00 32 28 1D

8、稳态滤波级数(12)

滤波强度（0~9）数值越大重量数值越稳定，但是加载反应越慢。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x000C
寄存器值	2 个字节	0x0000 至 0x0009
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x000C
寄存器值	2 个字节	0x0000 至 0x0009
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x00) 稳态滤波级数设置为 7 的实例：

请求：01 06 00 0C 00 07 08 0B

响应：01 06 00 0C 00 07 08 0B

9、AD 采样速率(13)

AD 采样速率越高，重量变化越快。

数值	0	1	2
速率	10bps	40bps	640bps

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x000D
寄存器值	2 个字节	0x0000 至 0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x000D
寄存器值	2 个字节	0x0000 至 0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) AD 采样速率设置为 40bps 的实例：

请求：01 06 00 0D 00 01 D9 C9

响应：01 06 00 0D 00 01 19 D8

10、通道号(14)

将当前的通道地址改为要设置的通道地址，通道地址范围（1-247）。

注：改完后将按照新的通道地址进行通信。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x000E
寄存器值	2 个字节	0x0000 至 0x00FF
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x0E
寄存器地址	2 个字节	0x000E
寄存器值	2 个字节	0x0000 至 0x00FF
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 通道号设置为 2 的实例：

请求：01 06 00 0E 00 02 69 C8

响应：01 06 00 0E 00 02 69 C8

11、波特率(15)

数值	0	1	2	3	4
波特率	9600	19200	38400	57600	115200

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x000F
寄存器值	2 个字节	0x0000 至 0x00FF
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06

寄存器地址	2 个字节	0x000F
寄存器值	2 个字节	0x0000 至 0x00FF
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 波特率设置为 115200 的实例：

请求：01 06 00 0F 00 04 B8 0A

响应：01 06 00 0F 00 04 B8 0A

12、通信格式(16)

数值	0
通信格式	8-n-1

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0010
寄存器值	2 个字节	0x0000 至 0x00FF
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0010
寄存器值	2 个字节	0x0000 至 0x00FF
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 通信格式设置为 8-n-1 的实例：

请求：01 06 00 10 00 00 88 0F

响应：01 06 00 10 00 00 88 0F

13、通信模式(17)

数值	0
通信模式	MODBUS RTU

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0011
寄存器值	2 个字节	0x0000 至 0x00FF
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0011

寄存器值	2 个字节	0x0000 至 0x00FF
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址(0x01)通信模式设置为 MODBUS RTU 的实例：

请求：01 06 00 11 00 00 D9 CF

响应：01 06 00 11 00 00 D9 CF

14、小数点位置(18)

所有重量信息（当前重量、分度值、最大量程）的小数点位置。

数值	0	1	2	3	4
小数点位置	无	1 位	2 位	3 位	4 位

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0012
寄存器值	2 个字节	0x00000 至 0xFFFFF
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0012
寄存器值	2 个字节	0x00000 至 0x0004
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址(0x01)小数点设置为 1 的实例：

请求：01 06 00 12 00 01 E8 0F

响应：01 06 00 12 00 01 E8 0F

15、最小分度值(19)

分度值可设置为 1、2、5、10、20、50，它决定了模块最小变化的数值。

它和小数点共同决定了模块的分辨率。例如单位为 kg，如果分度值为 2，小数点为 2 位，它的分辨率就是 0.02kg；如果分度值为 20，小数点为 3 位，它的分辨率就是 0.020kg。两者的分辨率的数值是一样的，但是后者多了一个无效零。

数值	1	2	10	20	50
分度值	1	2	10	20	50

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0013
寄存器值	2 个字节	1、2、10、20、50
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
------	-------	-----------

功能码	1 个字节	0x06
寄存器地址	2 个字节	0x0013
寄存器值	2 个字节	1、2、10、20、50
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 分度值设置为 2 的实例：

请求：01 06 00 13 00 02 F9 CE

响应：01 06 00 13 00 02 F9 CE

16、最大量程(20)

最大量程为 4 字节补码（占 2 个字），决定了当前重量的最大值，当重量大于最大量程 + 9×分度值时状态字中的超载标志就置为 1，此时用户系统应做出相应的报警提示，避免压坏秤体。

请求 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0014
寄存器数量	2 个字节	0x0002
字节数	1 个字节	0x04
寄存器值	4 个字节*	最大量程≤最小分度*100000
CRC16 校验码	2 个字节	低位在前，高位在后

响应 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0014
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 最大量程设置为 60000 的实例：

请求：01 10 00 14 00 02 04 00 00 EA 60 BC 18

响应：01 10 00 14 00 02 01 CC

17、有砝码零点标定(22)

标定零点，该命令修改设备零点参数。

请求 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0016
寄存器数量	2 个字节	0x0002
字节数	1 个字节	0x04
寄存器值	4 个字节*	任意非零值
CRC16 校验码	2 个字节	低位在前，高位在后

响应 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0016
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 进行零点标定的实例：

请求：01 10 00 16 00 02 04 ff ff ff ff 73 1d

响应：01 10 00 16 00 02 A0 0C

18、有砝码增益标定第一点(30)

标定加载点 1，该命令修改设备加载点 1 参数。

请求 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x001E
寄存器数量	2 个字节	0x0002
字节数	1 个字节	0x04
寄存器值	4 个字节*	目标重量
CRC16 校验码	2 个字节	低位在前，高位在后

响应 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x001E
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 加载点 1 设置为 500 的实例：

请求：01 10 00 1E 00 02 04 00 00 01 F4 73 38

响应：01 10 00 1E 00 02 21 CE

19、有砝码增益标定第二点(32)

标定加载点 2，该命令修改设备加载点 2 参数。

请求 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0020
寄存器数量	2 个字节	0x0002
字节数	1 个字节	0x04
寄存器值	4 个字节*	目标重量(第一点≤目标重量 ≤最大量程)
CRC16 校验码	2 个字节	低位在前，高位在后

响应 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0020
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 加载点 2 设置为 1000 的实例：

请求：01 10 00 20 00 02 04 00 00 03 E8 F1 09

响应：01 10 00 20 00 02 40 02

20、有砟码增益标定第三点(34)

标定加载点 3，该命令修改设备加载点 3 参数。

请求 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0022
寄存器数量	2 个字节	0x0002
字节数	1 个字节	0x04
寄存器值	4 个字节*	目标重量(第二点≤目标重量 ≤最大量程)
CRC16 校验码	2 个字节	低位在前，高位在后

响应 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0022
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 加载点 3 设置为 2000 的实例：

请求：01 10 00 22 00 02 04 00 00 07 d0 73 c2

响应：01 10 00 22 00 02 E1 C2

21、有砟码增益标定第四点(36)

标定加载点 4，该命令修改设备加载点 4 参数。

请求 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0024
寄存器数量	2 个字节	0x0002
字节数	1 个字节	0x04
寄存器值	4 个字节*	目标重量(第三点≤目标重量 ≤最大量程)
CRC16 校验码	2 个字节	低位在前，高位在后

响应 PDU

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x10
起始地址	2 个字节	0x0024
寄存器数量	2 个字节	0x0002
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 加载点 4 设置为 5000 的实例：

请求：01 10 00 24 00 02 04 00 00 13 88 FD 12

响应：01 10 00 24 00 02 01 C3

五、离散量

1、概述

离散量以位为单位，只能读取（0x01）。

寄存器地址 (十进制)	字数 (16bits)	功能说明	备注
0	1	稳定	
1	1	超载	
2	1	零位	
3	1	负数	

2、稳定

重量变化小于判稳范围后，稳定标志位为 1。

3、超载

当重量值大于满量程加 9 个分度值时，超载标志位为 1。当超载标志为 1 时，主机必须停止显示重量，并做出相应的报警提醒用户，避免压坏传感器。

4、零位

当重量值等于 0 时，零位标志位为 1。

5、负数

当重量值为负数时，负数标志位为 1。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x01
寄存器地址	2 个字节	0x0000
线圈数量	1 个字节	0x00-0x20
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x01
寄存器地址	1 个字节	0x000
线圈状态	1 个字节	0x000 或 0xFF
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个读取通道地址 (0x01) 状态的实例：

请求：01 01 00 00 00 08 3D CC

响应：01 01 01 05 91 8B(稳定、零位)

六、线圈

1、概述

线圈以位为单位，可读写 (0x01、0x05)，写入 0x0000 或 0xFF00 来触发某个功能 (开关量输出除外)。

寄存器地址 (十进制)	字数 (16bits)	功能说明	备注
4	1	进入 IAP 命令	模块响应后，需要上电重启，打开上位机进行刷机
6	1	上电自动清零	
9	1	全部复位	
10	1	校准复位	
11	1	参数复位	

2、进入 IAP 命令(4)

模块响应后，自动重启进入 boot 模式，等待打开厂商提供的上位机，进行刷机。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x0004
输出值	2 个字节	0x00000 或 0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x0004
输出器值	2 个字节	0x00000 或 0xFF00

CRC16 校验码	2 个字节	低位在前，高位在后
-----------	-------	-----------

这是一个通道地址(0x01)进入 IAP 命令的实例：

请求：01 05 00 04 FF 00 CD FB

响应：01 05 00 04 FF 00 CD FB

2、上电自动清零(6)

模块重新上电后，是否以当前重量作为零点，1：开启上电自动清零；0：关闭上电自动清零。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x0006
输出值	2 个字节	0x00000 或 0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x0006
输出器值	2 个字节	0x00000 或 0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址(0x01)开启上电自动清零的实例：

请求：01 05 00 06 FF 00 6C 3B

响应：01 05 00 06 FF 00 6C 3B

3、全部复位(9)

全部复位是指将校准参数和系统参数全部复位。

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x0009
输出值	2 个字节	0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x0009
输出器值	2 个字节	0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址(0x01)全部复位的实例：

请求：01 05 00 09 FF 00 5C 38

响应：01 05 00 09 FF 00 5C 38

4、校准复位(10)

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x000A
输出值	2 个字节	0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x000A
输出器值	2 个字节	0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 校准复位的实例：

请求：01 05 00 0A FF 00 AC 38

响应：01 05 00 0A FF 00 AC 38

5、参数复位(11)

请求

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x000B
输出值	2 个字节	0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

响应

通道地址	1 个字节	0x01-0xF7
功能码	1 个字节	0x05
输出地址	2 个字节	0x000B
输出器值	2 个字节	0xFF00
CRC16 校验码	2 个字节	低位在前，高位在后

这是一个将通道地址 (0x01) 参数复位的实例：

请求：01 05 00 0B FF 00 FD F8

响应：01 05 00 0B FF 00 FD F8